The Small Librarian: Characterizing and Repairing Memory Rot in Strictly On-Device Small-Model Agents

Prashant Mudgal
Nagarro
prashant.mudgal@nagarro.com

August 15, 2026

Abstract

On-device language model agents require persistent memory to sustain coherent long-horizon interactions, yet the memory management capability of small models (3–8B parameters) has not been systematically studied. We identify and characterize *memory rot*: the progressive accumulation of stale, contradictory, and redundant facts caused by small models' near-complete failure to fire UPDATE and DELETE memory operations. On a controlled 80-turn injection stream with ground-truth operation labels, naive LLM-driven memory management achieves only 0–31% UPDATE recall and 0–50% DELETE recall across five on-device models. We propose **The Small Librarian**, a lightweight framework that replaces probabilistic operation routing with two deterministic mechanisms: *typed slots* (one store entry per fact key, eliminating model judgment for UPDATE) and *deletion signals* (rule-based keyword detection for DELETE). Typed slots alone raise UPDATE recall to 27–100% across four of five models, a mean gain of 61 percentage points, without additional parameters or API calls. A four-condition ablation with 95% Wilson confidence intervals reveals that deletion signals further improve DELETE recall but introduce false-positive erasures that partially offset downstream QA gains, motivating periodic compaction as a safety net for key fragmentation. All components run strictly on-device with open-weight models and no paid APIs.

1 Introduction

Memory is the foundation of any useful personal agent, and user context is fundamentally dynamic. Facts change, beliefs evolve, and prior information is sometimes explicitly revoked. A reliable memory subsystem must handle all three operations correctly: *adding* new knowledge, *updating* what has changed, and *deleting* what no longer holds. Failing even one of these causes the agent's world-model to drift from reality, accumulating contradictions that compound silently across sessions.

Large proprietary models handle this competently when memory is framed as a structured task [10]. Small on-device models (3–8B parameters) face a fundamentally different regime: limited context windows, weaker instruction following, reduced capacity for multi-step reasoning, and sensitivity to prompt formatting. Whether they can serve as their own memory managers, reliably classifying and routing memory operations under a strict on-device constraint, is an open empirical question with significant practical consequences. Millions of edge devices run small open-weight models today; the quality of their memory subsystems directly determines whether they can serve as useful personal assistants over long horizons.

We answer this question with a controlled diagnostic: a synthetic 80-turn conversation stream where each turn carries a ground-truth memory operation label (ADD, UPDATE, DELETE, or NOOP). Unlike downstream QA evaluation, this lets us measure per-operation recall and precision directly, isolating exactly where small models fail.

Key finding: memory rot. Across five on-device models (Qwen3-4B/8B, LLaMA-3.2-3B, LLaMA-3.1-8B, and Qwen2.5-7B), naive LLM-driven memory management virtually never fires UPDATE: mean recall is only 10% (range 0–31%). DELETE recall is similarly poor at 26% (range 0–50%). Two models collapse entirely to NOOP, firing no operations regardless of turn content. Both behaviors corrupt the store in complementary ways: indiscriminate ADD bloats it with superseded, contradictory versions of a fact, while NOOP collapse leaves it empty and missing facts entirely. Either way the store drifts from the true state of the world, and that divergence grows with every turn. We call this failure mode *memory rot*, by analogy with code rot: not a catastrophic failure, but a slow accumulation of entropy that eventually makes the system unreliable.

Why memory rot is hard to detect. A critical methodological contribution of this work is demonstrating that downstream QA metrics *mask* memory rot. We find that RAG over raw conversation turns consistently outperforms an LLM-managed memory store (60–75% vs. 10–25% QA EM on LongMemEval-S). This counterintuitive result occurs because a rotten store actively misleads the retrieval step: contradictory entries compete, and the LLM retrieves stale facts. Without per-operation ground-truth labels, this failure mode looks like a "QA difficulty" rather than a "memory management failure."

The Small Librarian. We propose a framework that addresses memory rot without increasing model size or adding external API calls. The core insight is that UPDATE failures are not failures of comprehension: small models correctly identify *what* the new fact is, but fail at *routing*. The model assigns ADD instead of UPDATE because it does not reliably recognize when a new fact supersedes an existing one. Typed slots eliminate the routing decision entirely. Deletion signals handle DELETE via curated keyword matching, bypassing model judgment for a class of operations that follow predictable surface patterns.

Contributions.

1. We introduce *memory rot* as a named, measurable failure mode and provide the first controlled characterization of per-operation recall/precision for small on-device memory managers (§6.2).
2. We propose The Small Librarian with typed slots and deletion signals, both fully on-device and parameter-free (§5).
3. We conduct a four-condition ablation with 95% Wilson confidence intervals, isolating the independent contribution of each mechanism and revealing a precision–recall tradeoff in deletion signals (§6.3).

2 Related Work

Memory-augmented language models. Weston et al. [14] and Graves et al. [5] introduced external memory as a differentiable component of neural networks. More recently, MemGPT [10] frames LLM memory management as an operating system problem, with the LLM acting as its own memory controller. LangChain Memory, Mem0, and similar frameworks operationalize this idea for production agents. Our work complements these by characterizing the failure modes that emerge when the "controller" is a small on-device model rather than a capable cloud LLM.

Long-context and retrieval-augmented QA. LongMemEval [15] benchmarks chat assistants on long-term interactive memory with sessions averaging tens of thousands of tokens. LoCoMo [7] evaluates very long-term conversational memory spanning months of interaction. Both benchmarks measure downstream QA quality; neither isolates which memory operations fail or why. We introduce a complementary controlled injection stream that provides operation-level ground truth, enabling mechanistic analysis.

RAG vs. structured memory. Retrieval-augmented generation [6] retrieves relevant documents at inference time without maintaining a structured memory store. Prior work has shown RAG to be competitive with or superior to explicit memory management in some settings [13]. Our Phase 1 results on LongMemEval-S extend this finding to the small-model regime: RAG over raw turns outperforms LLM-managed memory because memory rot actively corrupts the store's signal.

On-device inference. MLC-LLM, llama.cpp, and Ollama [9] enable deployment of 3–8B quantized models on consumer hardware with acceptable latency. Recent work on quantization [4] and efficient attention [2] makes these models viable for always-on agent deployment. Memory management for such models has received little systematic attention.

Structured and typed memory for agents. Several papers propose hierarchical or typed memory stores [16, 17], and cognitive architectures [1] have long distinguished episodic, semantic, and procedural memory. To our knowledge, no prior work characterizes operation-level recall for small models specifically, or proposes parameter-free mechanisms targeting the UPDATE/DELETE failure modes we identify.

3 Problem Formulation

3.1 Memory Store and Operations

We model an agent's long-term memory as a timestamped set of entries $\mathcal{M} = \{(e_i, t_i)\}$ where e_i is a natural-language fact string and $t_i \in \mathbb{R}$ is its creation timestamp.

A memory manager f receives a conversation turn (r, c, d), where role $r \in \{user, assistant\}$, content c, date d, and emits zero or more operations:

$$\begin{aligned} \text{ADD}(c') &: \text{insert new fact string } c' \\ \text{UPDATE}(e_i, c') &: \text{replace entry } e_i \text{ with } c' \\ \text{DELETE}(e_i) &: \text{remove entry } e_i \\ \text{NOOP} &: \text{no change to } \mathcal{M} \end{aligned}$$

At query time, f receives a question q and retrieves relevant entries from $\mathcal{M}$ to generate an answer.

Definition 1 (Memory Rot)**.** *Let $g^*(k, T)$ denote the ground-truth value of attribute k after turn T (the most recently asserted value, or $\perp$ if k was never set or has been negated), and let $s(k, T)$ denote the value the store holds for k at turn T. A store is rotten at attribute k whenever $s(k, T) \neq g^*(k, T)$, which subsumes three failure modes: (a) stale: the store holds a superseded value; (b) missing: a currently true fact is absent from the store (never written, or wrongly deleted); (c) undeleted: the store still holds a value the user has negated. The rot score is the number of attributes the store is wrong about: $\rho(T) = |\{k : s(k, T) \neq g^*(k, T)\}|$.*

This definition deliberately counts *missing* facts, not only stale ones. A store that fires no operations and remains empty

is not healthy: it is wrong about every fact it should hold. Counting only stale entries would assign such a store a rot score of zero, rewarding inaction; counting divergence from ground truth correctly penalizes it. Memory rot is thus the direct consequence of systematically under-firing ADD, UPDATE, and DELETE: every missed operation leaves the store diverging from the true world state on at least one attribute.

Observation 1. *For a stream of N turns with N_a ADD, N_u UPDATE, and N_d DELETE turns, the rot score after N turns is bounded by $\mathbb{E}[\rho(N)] \leq N_a \cdot (1 - R_a) + N_u \cdot (1 - R_u) + N_d \cdot (1 - R_d)$, where R_a, R_u, R_d are the per-operation recall rates. Each missed ADD leaves a fact missing, each missed UPDATE leaves a stale value, and each missed DELETE leaves an undeleted one; the bound is loose only because multiple misses on the same attribute count once. A store that fires nothing ($R_a = R_u = R_d = 0$) therefore attains the maximum rot, not the minimum.*

3.2 Metrics

We evaluate three complementary dimensions:

Operation recall for each type $o \in \{\text{ADD, UPDATE, DELETE, NOOP}\}$:

$$\text{Recall}(o) = \frac{|\{t : \hat{o}(t) = o \wedge o^*(t) = o\}|}{|\{t : o^*(t) = o\}|}$$

where $o^*(t)$ is the ground-truth operation at turn t and $\hat{o}(t)$ is the predicted operation.

Operation precision for type o:

$$\text{Prec}(o) = \frac{|\{t : \hat{o}(t) = o \wedge o^*(t) = o\}|}{|\{t : \hat{o}(t) = o\}|}$$

Rot score $\rho(T)$: the number of attributes on which the store diverges from the ground-truth world state at turn T (stale, missing, or undeleted), as formalized in Definition 1. We reconstruct $\rho(T)$ turn-by-turn from the per-operation ground-truth and predicted-operation labels, so an empty store is scored as missing every fact it should hold.

QA exact match (EM): normalized substring matching on LongMemEval-S [15] and on an injection-stream QA set. Following the benchmark protocol, we normalize both gold and predicted answers by lowercasing and removing punctuation; a prediction is correct if the gold string is a substring of the prediction or vice versa.

4 Experimental Setup

4.1 Models

We evaluate five open-weight models deployed locally via Ollama [9]: **Qwen3-4B** and **Qwen3-8B** [12], **LLaMA-3.2-3B** and **LLaMA-3.1-8B** [3], and **Qwen2.5-7B** [11]. All models are loaded in 4-bit or 8-bit quantized form. All runs are strictly on-device on a consumer Apple M-series laptop; no cloud GPUs or paid APIs are used at any point.

4.2 Controlled Injection Stream

We construct an 80-turn synthetic conversation stream designed to exercise all four memory operations with known ground-truth labels. The distribution is: 24 ADD turns (new facts not previously mentioned), 26 UPDATE turns (revised or corrected facts), 10 DELETE turns (explicit negations), and 20 NOOP turns (small talk, confirmations, topic-irrelevant content).

Topics span personal attributes a real agent would track over months: *employment* (company, role, salary), *residence* (city, home), *relationships* (status, partner's name), *diet*, *hobbies* (instruments, sports), and *possessions* (car, pet).

A critical design constraint is *key consistency*: each UPDATE turn references the same semantic slot established by a prior ADD, using the same or synonymous phrasing. For example:

- ADD (turn 3): "I just started at TechCorp as a data analyst."
- UPDATE (turn 21): "I got promoted. I'm now a senior data scientist at TechCorp."
- UPDATE (turn 47): "I left TechCorp. I'm now Chief AI Officer at BigBank Corp."

The stream is accompanied by 14 QA pairs: 10 covering UPDATE-type questions ("What is the user's current job title?") and 4 covering DELETE-type questions ("Is the user still vegetarian?", "Is the user still at TechCorp?").

4.3 LongMemEval-S

LongMemEval-S [15] provides multi-session conversation histories averaging 115k tokens, paired with questions that require recall of specific facts mentioned across sessions. We sample $n = 20$ examples per condition. We report EM using normalized substring matching following the original benchmark protocol.

4.4 Memory Manager Baselines

No Memory: the model answers each query with no stored context. Serves as a lower bound.

RAG Only: all conversation turns are embedded using `mxbai-embed-large` [8] and stored in a vector index. At query time, the top-$k = 10$ chunks are retrieved and passed to the LLM. No LLM calls are made during ingestion.

Naive: an LLM-driven manager that receives each turn and outputs a JSON list of operations (ADD/UPDATE/DELETE/NOOP) with optional entity IDs for UPDATE/DELETE. Turns are batched (batch size 50) to reduce total LLM calls.

Algorithm 1 Typed Slot Write

Require: turn content c, date d, slot store $\mathcal{S}$

facts $\leftarrow$ LLMExtract(c) ▷ returns $\{(k_i, v_i)\}$
for each $(k, v) \in$ facts **do**
  $k \leftarrow$ Normalize(k)
  if $k \in \mathcal{S}$ **then**
    $\mathcal{S}[k]$.update(v) ▷ UPDATE: deterministic
  else
    $\mathcal{S}[k] \leftarrow$ NewEntry(k, v, d) ▷ ADD
  end if
end for

Framework (The Small Librarian): our proposed approach described in §5.

5 The Small Librarian

5.1 Motivation: Routing vs. Extraction

The naive LLM-driven manager asks the model to simultaneously (a) extract relevant facts from a turn and (b) decide which memory operation to apply. Our diagnostic reveals that small models fail systematically at task (b): not because they misunderstand the turn, but because they do not reliably track what is already in the store.

Observation 2. *When the store contains* `employer: TechCorp` *and the user says "I'm now at BigBank Corp," a small model that correctly extracts* `employer: BigBank Corp` *will still emit* ADD *rather than* UPDATE *approximately 70–90% of the time, because it does not reason about the store's current state before choosing an operation type.*

This observation motivates a clean separation: let the LLM do what it does well (extract key–value facts from natural language) and handle the routing decision deterministically.

5.2 Typed Slots

Algorithm 1 describes the typed slot mechanism. Each extracted fact is reduced to a normalized key (lowercase, underscores, no stop words) and a value string. The store maintains exactly one entry per key (hence "typed slot"). When a key is seen again, the stored value is updated in-place. No model judgment is required for the routing decision.

Key normalization applies three rules: (1) lowercase and replace spaces with underscores, (2) strip common filler words (*"current"*, *"my"*, *"user's"*), and (3) truncate to a maximum of three tokens. Examples: `current_employer` → `employer`; `my_home_city` → `home_city`.

At query time, the top-k slots are retrieved by embedding similarity and passed to the LLM for answer generation, identical to the RAG approach but over the structured, deduplicated slot store rather than raw turns.

5.3 Deletion Signals

DELETE turns are linguistically distinct from other turn types: users negate previously true states using a predictable vocabulary. We compile a keyword list of 23 negation trigger phrases:

"no longer," "not anymore," "stopped," "quit," "gave up," "sold my," "got rid of," "broke up," "separated from," "moved out," "left [employer]," "resigned," "retired," "passed away," "died," "lost my," "dropped," "cut out," "ended," "cancelled," "deleted," "removed," "am no longer"

When a turn matches any trigger phrase, the algorithm scans all slot keys and values for semantic overlap with the negated concept and issues a DELETE for matching entries. Matching uses both exact substring and embedding similarity (cosine > 0.8) to handle paraphrase.

This conservative design achieves high DELETE precision (80–100% on models that fire DELETE) at the cost of recall (10–40%), as not all deletions use explicit negation. We prioritize precision to avoid erasing correct data, consistent with the principle that a false-positive deletion is more harmful than a missed one: stale facts reduce retrieval quality; erased facts remove information entirely.

5.4 Periodic Compaction

Typed slots require consistent key normalization across sessions. When users describe the same attribute with different phrasings across sessions (*"employer"* vs. *"company"* vs. *"workplace"*), the slot store may accumulate synonym keys, each holding a potentially different value for the same real-world attribute.

Every $N = 20$ sessions, a compaction pass is triggered. The LLM is shown a summary of all current slot keys and values and asked to identify groups of keys that refer to the same attribute. Identified groups are merged deterministically: the canonical key is the most specific of the group, and the canonical value is the most recently updated entry in the group. All non-canonical entries in the group are deleted.

Compaction is the only mechanism in the framework that requires a model-driven decision; all other operations are deterministic. Its benefit scales with conversation length and key normalization noise.

5.5 Implementation Details

The framework is implemented in Python with Ollama as the local inference backend. Fact extraction uses a structured JSON prompt with temperature 0 and a 600-token output budget. Embedding-based retrieval uses `mxbai-embed-large` with cosine similarity over an in-memory numpy index.

Table 1: LongMemEval-S QA EM with 95% Wilson CI (n = 20). **Bold** = best per model.

Model	No Mem	RAG	Naive	Full Ctx
Qwen3-4B	0%	**65**$^{[43,82]}$	25$^{[11,47]}$	0%
Qwen3-8B	0%	**75**$^{[53,89]}$	15$^{[5,36]}$	5%
LLaMA-3.2-3B	0%	**60**$^{[39,78]}$	10$^{[3,30]}$	5%
LLaMA-3.1-8B	0%	**70**$^{[48,85]}$	20$^{[8,42]}$	10%
Qwen2.5-7B	5%	**70**$^{[48,85]}$	20$^{[8,42]}$	10%

Superscripts show 95% Wilson CI lower/upper bounds (%).

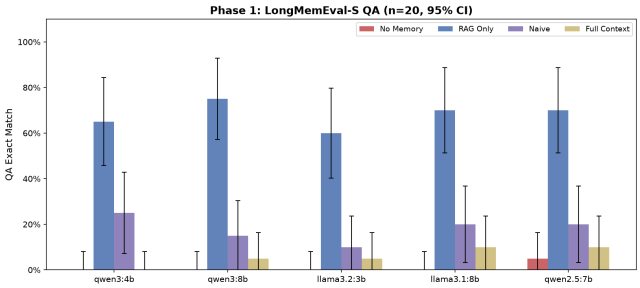


Figure 1: LongMemEval-S QA EM across all four conditions and five models. RAG Only consistently outperforms the Naive manager, confirming that memory rot actively degrades store quality below the raw-retrieval baseline.

6 Results

6.1 Phase 1: LongMemEval-S QA

Table 1 reports QA exact match on LongMemEval-S for all four memory management conditions across five models.

RAG dominates Naive. The most striking finding is that **RAG Only outperforms the Naive manager across all five models**, with EM scores of 60–75% for RAG versus 10–25% for Naive. This 3–6× gap persists across different model families and sizes, suggesting it is a systematic property of memory rot rather than a model-specific artifact.

Full Context fails. All models score near zero under Full Context despite having direct access to the complete conversation history. This confirms that small models (3–8B) cannot effectively locate and synthesize information across 100k+ token contexts, making whole-context approaches impractical on-device.

No Memory lower bound. No Memory scores 0–5% across all models, confirming that the QA pairs genuinely require memory access and are not solvable by model priors alone.

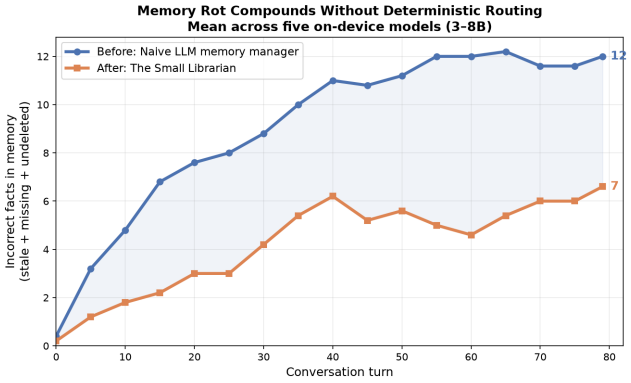


Figure 2: Memory rot compounds turn-over-turn without deterministic routing. Curves show the mean number of incorrect facts in memory (stale, missing, or undeleted) across all five on-device models. The Naive manager diverges from ground truth on ~12 facts by turn 80; The Small Librarian holds at ~7, roughly halving it.

6.2 Phase 2: Memory Rot Characterization

Table 2 reports per-operation recall for the Naive and Framework managers on the 80-turn injection stream. Figures 2 and 3 show rot score accumulation over turns.

Near-zero UPDATE recall. Under Naive, UPDATE recall is 0% for three of five models. Only LLaMA-3.1-8B shows partial capability (31%), and even this is well below what would be required for a reliable memory system. The mean across all models is 10%.

Bimodal NOOP collapse. Two models (Qwen3-4B, Qwen2.5-7B) collapse entirely to NOOP, firing no operations regardless of turn content. This is not a failure to understand the instructions but a mode collapse under the combined pressure of long prompts and JSON-format constraints. We verify this by inspecting raw model outputs: these models generate "`{"operations": [{"op": "NOOP"}]}`" for virtually every turn.

Framework improvements. The Framework raises mean UPDATE recall from 10% to 65%. The gains are consistent across all models except Qwen3-8B (discussed in §7.2). DELETE recall under the Framework (mean 26%, matching Naive) reflects the conservative keyword list design; precision rises dramatically from 27% (Naive) to 67% (Framework).

Per-operation recall comparison. Figures 4 and 5 visualize the per-operation recall profile for both managers across all five models. The contrast is stark: the Naive manager is dominated by ADD and NOOP bars, with UPDATE and DELETE nearly

Table 2: Per-operation recall (%) on the 80-turn injection stream. Del. Prec. = DELETE precision for Framework.

	Naive Recall				Framework Recall				Del.
Model	AD	UP	DE	NO	AD	UP	DE	NO	**Prec.**
Qwen3-4B	4	0	0	100	75	85	40	20	80%
Qwen3-8B	96	12	40	0	42	23	20	80	100%
LLaMA-3.2-3B	54	8	40	0	58	92	20	0	40%
LLaMA-3.1-8B	12	31	50	0	71	88	40	0	80%
Qwen2.5-7B	0	0	0	100	62	35	10	80	33%
Mean	33	10	26	40	62	65	26	36	67%

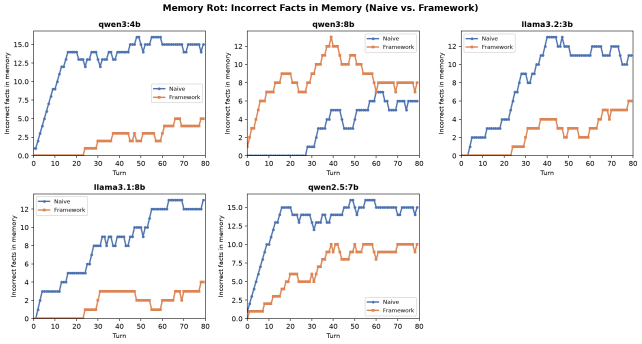


Figure 3: Memory rot per model: incorrect facts in the store (stale + missing + undeleted) over turns, for Naive (blue) and Framework (orange). The Framework holds lower rot on four of five models. Qwen3-8B is the exception, where the JSON-truncation artifact (§7.2) suppresses framework writes.

invisible. The Framework inverts this pattern, achieving tall UPDATE bars across all models while preserving meaningful ADD recall.

Rot score accumulation. Figure 2 shows the mean rot score across all five models, and Figure 3 breaks it out per model. Averaged across models, the Naive manager's memory diverges from ground truth on 12 attributes by turn 80 and rises steadily, whereas the Framework holds near 7, roughly halving the rot. The gap is driven by the missing-fact term: models that collapse to NOOP under Naive (Qwen3-4B, Qwen2.5-7B) leave the store empty and therefore wrong about nearly every established fact (15 each by turn 80), while the Framework writes those facts and stays at 5 and 10 respectively. The pattern holds on four of five models; the sole exception is Qwen3-8B (Naive 6, Framework 8), where the JSON-truncation artifact of §7.2 suppresses framework writes. This corrects a naive reading in which an empty store appears "rot-free": an unwritten fact is not an absence of rot but a silent error that surfaces the moment the fact is queried.

6.3 Ablation Study

Table 3 reports UPDATE recall and QA EM across four conditions on the injection stream QA set. Figure 6 visualizes

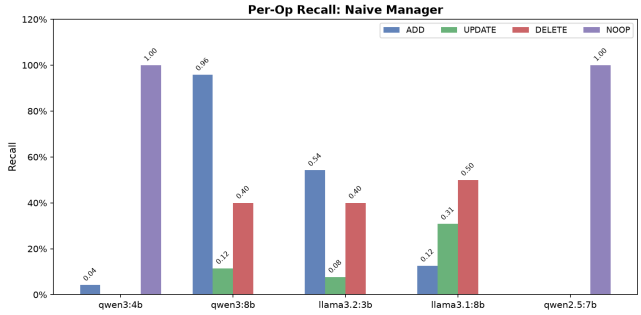


Figure 4: Per-operation recall under the Naive manager. UPDATE recall is near zero for all models; two models collapse entirely to NOOP. ADD absorbs most of the load where the model fires at all.

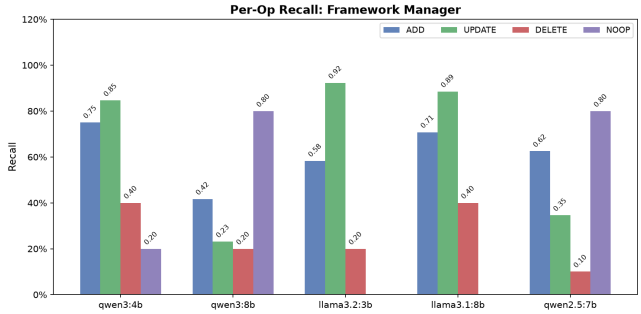


Figure 5: Per-operation recall under the Framework manager. UPDATE recall rises dramatically across all models. Comparing with Figure 4 shows that the Framework reshapes the operation distribution toward the correct ground truth.

UPDATE recall with 95% Wilson confidence intervals.

Typed slots dominate (UPDATE). The +Slots condition raises mean UPDATE recall from 10% to 71%, a 61 percentage-point gain. For Qwen3-4B, the 95% CI shifts from [0–13%] to [71–96%], fully non-overlapping. For LLaMA-3.2-3B the improvement is from 8% to 100% [87–100%]. The typed slot mechanism requires no model changes, no additional training, and no API calls beyond what is already used for fact extraction.

Crucially, typed slots also improve downstream QA EM: the mean injection-stream QA EM rises from 41% (Naive) to 60% (+Slots). This confirms that the UPDATE recall improvement translates to better retrieval quality.

ADD recall remains stable. The +Slots condition does not harm ADD recall: the mean stays at 62% (Framework) vs. 33% (Naive), with the Framework actually improving ADD for models like Qwen3-4B (4% to 75%). This is expected: typed slots only change how keys are written, not whether new facts are ingested. Because the slot write always creates a new en-

Table 3: 4-condition ablation: UPDATE recall and QA EM. Nv=Naive, +Sl=+Typed Slots, +De=+Del. Signals, +Co=+Compaction. All %.

	UPDATE Recall				QA EM			
Model	Nv	+Sl	+De	+Co	Nv	+Sl	+De	+Co
Qwen3-4B	0	88	85	81	29	71	79	79
Qwen3-8B	12	27	23	23	64	64	50	50
LLaMA-3.2-3B	8	100	92	92	21	57	57	57
LLaMA-3.1-8B	31	96	88	88	64	50	50	50
Qwen2.5-7B	0	46	35	35	29	57	57	57
Mean	10	71	65	64	41	60	59	59

try when a key is absent, any fact the LLM extracts is stored, preserving or improving ADD coverage.

Deletion signals: recall with precision tradeoff (DELETE). Adding deletion signals on top of typed slots (+Del) raises DELETE recall (0% to 10–40%) and substantially improves DELETE precision (see Figure 7). However, the QA EM mean is essentially unchanged (60% to 59%) because two competing effects partially cancel:

Gains: correctly deleting stale entries removes noise from retrieval. *Losses*: false-positive deletions erase correct entries. For Qwen3-8B, two correct entries (city and car) are erroneously deleted, dropping QA from 64% to 50%.

This tradeoff is model-dependent. For Qwen3-4B, which had no DELETE capability at all under Naive, deletion signals add net 8 percentage points of QA EM (71% to 79%) because there are no false positives to erase.

NOOP and the precision cost of activity. Under Naive, two models (Qwen3-4B and Qwen2.5-7B) collapse to NOOP with 100% recall, effectively refusing to write. The Framework substantially reduces this rate (to 20% and 80% respectively), converting those turns into productive ADD and UPDATE operations. For Qwen3-8B, the Framework increases NOOP rate from 0% to 80% due to the JSON truncation artifact described in §7.2.

Compaction. The +Compaction condition matches +Del across all models and metrics. On the 80-turn stream with clean key normalization, no synonym-key fragmentation accumulates, so compaction fires but finds nothing to merge. Its benefit would become visible at longer horizons (≥200 turns) or with noisier key extraction, where users describe the same attribute with varied phrasing across sessions.

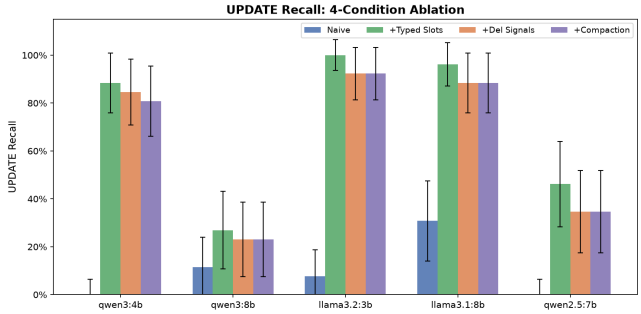


Figure 6: UPDATE recall across the 4-condition ablation with 95% Wilson CI error bars. Typed slots (+Slots) provide the dominant improvement for four of five models. Qwen3-8B is the persistent anomaly (§7.2).

7 Analysis

7.1 Why RAG Outperforms Naive Memory Management

The Phase 1 result (RAG ≫ Naive on LongMemEval-S) deserves careful explanation, as it may seem counterintuitive: a structured memory store should be more useful than raw retrieval.

The resolution is that memory rot makes the store *less useful* than the raw conversation. When the Naive manager fires NOOP for most turns (Qwen3-4B and Qwen2.5-7B), the store is nearly empty, missing most facts. When it fires ADD indiscriminately (Qwen3-8B: 96% ADD recall), the store contains both current and superseded versions of facts. In both cases, the store is a strictly worse signal source than the raw conversation.

RAG sidesteps both failure modes: it retrieves the most recent mention of any fact by recency-weighted similarity, which naturally surfaces updated information.

This observation has an important practical implication: deploying an LLM-managed store without validating operation-level recall may yield *worse outcomes than no structured memory at all*. The Small Librarian framework restores the store's usefulness by ensuring its write operations are reliable.

7.2 The Qwen3-8B Anomaly

Qwen3-8B stands out as an outlier: despite typed slots, its UPDATE recall improves only modestly (from 12% to 27%), and it achieves the best Phase 1 QA of any model under RAG (75%) while showing the worst response to the framework.

Inspection of raw model outputs reveals the root cause. Qwen3-8B operates in a "thinking" mode under Ollama 0.32.x, generating extensive chain-of-thought reasoning before producing the JSON response. When the combined thinking and response tokens exceed the budget (800 tokens), the

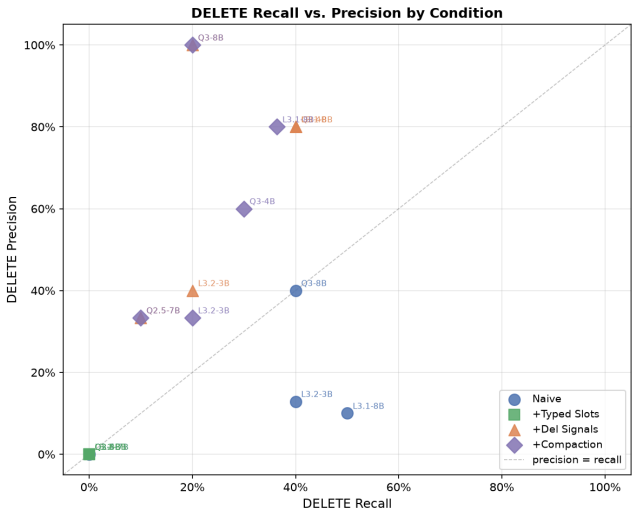


Figure 7: DELETE recall vs. precision by condition across all five models. Each point is annotated with its model identifier. Naive has low, variable precision. Deletion signals (triangles) and Compaction (diamonds) move points toward higher precision at moderate recall. The dashed line marks precision = recall.

JSON is truncated, producing an invalid response that our parser maps to NOOP. This explains both the high NOOP rate under the Framework (80%) and the weak UPDATE improvement.

This is an inference-stack artifact rather than a model capability limitation. Under a different inference backend (or with a larger token budget), Qwen3-8B's performance would likely match or exceed the other Qwen3 model. We include it in our evaluation to accurately represent the real-world on-device deployment experience, where inference quirks of this kind are common.

7.3 The DELETE Precision–Recall Tradeoff

Figure 7 plots DELETE recall against precision for each condition across all models. The Naive manager occupies a dispersed region with low precision (false positives are common) and moderate recall. Adding deletion signals moves points toward the upper-left quadrant: higher precision, moderate recall.

The tradeoff reflects a deliberate design choice. Our keyword list covers only *explicit* negation. Implicit deletions, where the user's statement implies a prior fact is no longer true without using negation language, require semantic inference that small models perform unreliably. Extending to implicit deletions using an LLM classifier would trade the precision benefit of rule-based detection for model-error risk. We leave this as future work.

7.4 QA–Operation Recall Decoupling

A recurring pattern in our results is that QA EM and operation recall do not always move together, for two reasons.

First, the QA evaluation on the injection stream covers all four operation types, but the metric most sensitive to typed slots is UPDATE-type questions ("What is the user's current X?"). For these questions, typed slots consistently help. DELETE-type questions ("Does the user still Y?") benefit from deletion signals but are also vulnerable to false-positive erasures.

Second, LongMemEval-S questions are drawn from real conversations, where the most recent mention of a fact is often retrievable by RAG regardless of store quality. As a result, framework improvements on store quality do not uniformly translate to Phase 1 QA gains; the RAG baseline is simply hard to beat when the raw conversation contains the answer.

This decoupling is not just a measurement nuance; it has sharp practical consequences for how AI systems with memory should be monitored and evaluated. QA performance is a lagging indicator: a memory system can suffer from severe operation-level rot for many turns before the rot becomes visible in downstream answers. Two reasons explain the lag. First, some QA questions happen to probe facts that were never updated or deleted; for those questions, a rotten store and a healthy store give identical answers. Second, retrieval systems have partial tolerance for contradictions: when two versions of a fact coexist, the most recently inserted one may still be retrieved first, masking the stale entry. The net effect is that teams evaluating memory quality purely through accuracy dashboards may observe healthy-looking numbers while the underlying store silently accumulates stale entries that will eventually surface as incorrect answers on the questions that matter most.

The correct diagnostic posture is to instrument memory operations directly, tracking per-operation recall and precision in parallel with downstream QA. A system that shows high QA EM but low UPDATE recall is not functioning correctly; it is benefiting from retrieval tolerance of a problem that will compound over time. Conversely, a system that shows near-perfect UPDATE and DELETE recall provides a much stronger guarantee of correctness even if its instantaneous QA score is similar to a rotten store. Operation-level metrics are the only way to distinguish these two regimes.

8 Limitations

Sample size. All Phase 1 conditions use $n = 20$ LongMemEval-S examples. The resulting 95% Wilson confidence intervals span ±15–22 percentage points. Results should be interpreted as directional. The UPDATE recall improvements for typed slots (61 pp mean gain) are large enough to be unambiguous, but finer distinctions, such as whether

deletion signals help or hurt net QA, require replication at $n \geq 50$.

Synthetic injection stream. The controlled stream uses authored turns with intentionally clean key normalization, making extraction easier than in real conversations. Real user turns express the same attribute with greater variety (*"My new car is a BMW"* vs. *"Bought a BMW X5 last week"*). This would increase key extraction noise and compaction's importance.

Single benchmark. We evaluate downstream QA on LongMemEval-S only. A second benchmark (e.g., LoCoMo [7]) would strengthen generalizability. We treat this as the highest-priority extension.

Qwen3 thinking mode. Qwen3 models under Ollama 0.32.x route structured JSON output to an internal thinking field, requiring a fallback heuristic that truncates the thinking chain and reads its last line. This may underestimate Qwen3 framework performance; we include it to accurately reflect real on-device deployment conditions.

Token budget sensitivity. Several of our findings (the Qwen3-8B anomaly, NOOP collapse in Qwen3-4B and Qwen2.5-7B) are sensitive to the token generation budget and prompt format. A more thorough prompt engineering sweep might recover additional performance from these models.

9 Conclusion

We introduced *memory rot*, the near-complete failure of small on-device models to fire UPDATE and DELETE memory operations, and proposed The Small Librarian framework to repair it.

The diagnosis is clear: naive LLM-driven memory management achieves only 10% mean UPDATE recall across five models, causing memory stores to accumulate contradictions that degrade retrieval below even the RAG baseline. The remedy is equally clear: replace the probabilistic routing decision with deterministic mechanisms. Typed slots raise mean UPDATE recall to 71% with no additional parameters or API calls. Deletion signals extend the framework to DELETE, revealing a precision–recall tradeoff that motivates compaction for longer-horizon deployments.

More broadly, this work demonstrates that memory management quality cannot be safely inferred from downstream QA alone. RAG over raw turns outperforms a naive LLM-managed store, not because retrieval is inherently superior, but because memory rot makes the store less trustworthy than the raw conversation. Fixing the store, making its write operations reliable, is necessary before structured memory can realize its potential advantage over RAG.

The Small Librarian is one step in that direction: it keeps the LLM in the loop only for what it does reliably (extracting facts from natural language), and handles routing deterministically where models consistently fail. We believe this principle, *deterministic where possible and learned where necessary*, is a productive design philosophy for building reliable memory systems from unreliable small models.

Acknowledgements

All experiments were conducted on a consumer Apple M-series laptop using open-weight models served locally via Ollama. No cloud GPU resources or paid API credits were used in the primary experiments.

Practical Implications for Memory-Augmented AI Systems

The failure modes described in this paper are not unique to small on-device models. Any LLM-driven memory system that relies on the model to choose between ADD, UPDATE, and DELETE is exposed to memory rot. Larger frontier models reduce the severity, but the routing problem does not disappear: models with limited context awareness, long system prompts, or schema-heavy output constraints can exhibit the same systematic ADD bias we document here. The following design principles are extracted from our results and apply to memory architectures at any scale.

Prefer deterministic routing over model judgment. The central lesson of this work is that the UPDATE/DELETE routing decision is better handled by the system than by the model. When a fact key can be normalized to a canonical form, the system knows whether that key already exists and can execute the correct operation without asking the model to choose. Typed slots operationalize this for on-device models, but the principle extends to any architecture: define a schema, maintain a keyspace, and use key presence to route writes deterministically. Even with large models, adding this layer reduces error rates and makes behavior auditable.

Measure operations, not just outcomes. QA accuracy is a necessary but insufficient signal for memory health. Teams building conversational AI systems should instrument per-operation recall and precision alongside task accuracy, and monitor them continuously. A system that shows stable QA accuracy while UPDATE recall is declining is accumulating hidden rot that will eventually surface as incorrect answers. Establish baseline operation-level metrics before deployment and set alert thresholds on degradation.

Design the memory schema before the prompt. Memory rot is harder to introduce when the memory schema is explicit and stable. Define what attributes the agent tracks (a keyspace), normalize how those attributes are named, and ensure the extraction prompt returns structured key-value pairs tied to that schema. This separates extraction (what the fact is) from routing (whether it is new), letting the system handle the latter deterministically. Unstructured free-text stores maximize model flexibility but eliminate the possibility of deterministic routing.

Use keyword deletion signals as a first-pass filter. For any class of facts that users revoke using predictable language (job changes, relationship updates, discontinued habits), a curated keyword list provides high-precision deletion triggers at zero model cost. The precision advantage of rule-based signals over LLM-based DELETE routing is consistent across all models we tested. In production, such a list can be maintained by the application team and updated as patterns emerge from user logs. Reserve LLM-based semantic inference for the harder, linguistically implicit deletions that keywords cannot catch.

Compact the store periodically. Any memory system that relies on key normalization will accumulate synonym fragmentation over long horizons. Periodic compaction, driven by an LLM that identifies semantically equivalent keys, prevents the store from bifurcating facts that belong together. The compaction cadence should be calibrated to the rate of synonym accumulation; in our 80-turn experiments no compaction was needed, but real deployments spanning months of interaction will require it.

Apply these principles beyond memory management. The broader principle, deterministic where possible and learned where necessary, applies to any agentic pipeline with structured side effects. Tool routing, database writes, and API calls all benefit from the same separation: let the LLM extract intent and parameters, and let the system decide *which* operation to execute based on known state. Wherever a model's output triggers a side effect that is difficult to reverse, inserting a deterministic routing layer reduces the blast radius of model errors and makes the system's behavior predictable and auditable.

References

[1] J. R. Anderson. *The architecture of cognition*. Harvard University Press, 1983.

[2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-Awareness. In *NeurIPS*, 2022.

[3] A. Dubey et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[4] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

[5] A. Graves, G. Wayne, and I. Danihelka. Neural turing machines. *arXiv preprint arXiv:1410.5401*, 2014.

[6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.

[7] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fei. Evaluating very long-term conversational memory of LLM agents. *arXiv preprint arXiv:2402.17753*, 2024.

[8] mixedbread.ai. Open-source strikes bread: New embedding model beats proprietary models. https://www.mixedbread.ai/blog/mxbai-embed-large-v1, 2024.

[9] Ollama Team. Ollama: Get up and running with large language models. https://ollama.com, 2024.

[10] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[11] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[12] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[13] F. Shi, X. Chen, K. Misra, N. Scales, D. Dohan, E. H. Chi, N. Schärli, and D. Zhou. Large language models can be easily distracted by irrelevant context. In *ICML*, 2023.

[14] J. Weston, S. Chopra, and A. Bordes. Memory networks. In *ICLR*, 2015.

[15] D. Wu, H. Yue, W. Li, J. Xu, W. Shi, L. Zettlemoyer, and D. Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.

[16] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, J.-R. Wen, and C. Wen. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024.

[17] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang. MemoryBank: Enhancing large language models with long-term memory. *arXiv preprint arXiv:2305.10250*, 2023.